

Practical 3

Aim: Implement Word Count / Frequency Program using MapReduce.

Problem Definition:

A digital publishing company stores thousands of news articles and blog posts in a Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations. As a Big Data Engineer, you are required to develop a MapReduce application that processes the text data and generates word frequency statistics.

Objectives:

To configure the Hadoop environment and upload text data to HDFS, design and implement a Mapper and Reducer for word counting, execute the MapReduce job on the Hadoop cluster, analyze the resulting word frequency output to identify the top 10 most frequent words, and evaluate how processing time scales with dataset size.

Observation / Output (Screenshots or command outputs)

1. Verify Hadoop services are running (prerequisite check).

```
jps
```

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu-Coursework]
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ jps
5408 NameNode
6049 ResourceManager
6181 NodeManager
5781 SecondaryNameNode
6535 Jps
5560 DataNode
```

2. Task 1 — Create the input directory and upload the text dataset to HDFS.

Create input.txt on the local file system with the sample dataset:

```
Hello world
hello haddoop
world hadoop
hello big analytics
big data analyticsSSS
```

```
hdfs dfs -mkdir -p /user/hadoop/wordcount/input
hdfs dfs -put input.txt /user/hadoop/wordcount/input
hdfs dfs -ls /user/hadoop/wordcount/input
```

Output:

```
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu-Coursework]
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ jps
5408 NameNode
6049 ResourceManager
6181 NodeManager
5781 SecondaryNameNode
6535 Jps
5560 DataNode
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ mkdir ~/MapReduceLab
mkdir: /home/aubaid-ahmed/MapReduceLab: File exists
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cd ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ pwd
/home/aubaid-ahmed/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ nano input.txt
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -mkdir -p /user/$USER/wordcount/input
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount
Found 1 items
drwxr-xr-x  - aubaid-ahmed supergroup          0 2026-09-11 09:26 /user/aubaid-ahmed/wordcount/input
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -put input.txt /user/$USER/wordcount/input/
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount/input
Found 1 items
-rw-r--r--  2 aubaid-ahmed supergroup          79 2026-09-11 09:27 /user/aubaid-ahmed/wordcount/input/input.txt
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/input/input.txt
Hello world
hello haddoop
world hadoop
hello big analytics
big data analytics
```

3. Task 2 — Develop the Mapper program (mapper.py) using Hadoop Streaming.

Since the job is implemented in Python, it runs through the Hadoop Streaming utility, which lets any executable that reads from standard input and writes to standard output act as the Mapper or Reducer. The Mapper reads the input text line by line from stdin, splits each line into words on whitespace, converts them to lowercase, and prints an intermediate key-value pair (word, 1) for every word encountered.

Code:

The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the file structure of a project named 'BDA-SHP-2026', which includes folders 'Practical1', 'Practical2', and 'Practical3'. Inside 'Practical3', there are files 'input.txt', 'mapper.py', 'reducer.py', and 'README.md'. The 'mapper.py' file is currently selected and open in the main editor. The code in 'mapper.py' is as follows:

```

1  #!/usr/bin/env python3
2
3  import sys
4
5  for line in sys.stdin:
6      words = line.strip().split()
7
8      for word in words:
9          print(word.lower() + "\t1")

```

The status bar at the bottom indicates the current line and column (Ln 9, Col 36), encoding (UTF-8), and the Python interpreter (Python 3.13 (64-bit)).

4. Task 3 — Develop the Reducer program (reducer.py) using Hadoop Streaming.

Hadoop Streaming sorts the Mapper's output by key before handing it to the Reducer, so all occurrences of the same word arrive as consecutive lines on stdin. The Reducer tracks the current word, accumulates its count, and prints the final (word, total-count) pair whenever the key changes.

Code:

The screenshot shows the Visual Studio Code editor interface with the 'reducer.py' file selected and open in the main editor. The Explorer panel on the left shows the same file structure as the previous screenshot, with 'reducer.py' now selected. The code in 'reducer.py' is as follows:

```

1  #!/usr/bin/env python3
2
3  import sys
4
5  current_word = None
6  current_count = 0
7
8  for line in sys.stdin:
9      word, count = line.strip().split("\t")
10     count = int(count)
11
12     if current_word == word:
13         current_count += count
14     else:
15         if current_word is not None:
16             print(current_word + "\t" + str(current_count))
17
18             current_word = word
19             current_count = count
20
21 if current_word is not None:
22     print(current_word + "\t" + str(current_count))

```

The status bar at the bottom indicates the current line and column (Ln 9, Col 36), encoding (UTF-8), and the Python interpreter (Python 3.13 (64-bit)).

5. Make the scripts executable and test them locally before running on Hadoop.

It is good practice to test the mapper/reducer pipeline locally with plain shell pipes before submitting it as a Hadoop job — this catches syntax and logic errors without the overhead of launching a cluster job.

```
chmod +x mapper.py reducer.py

# local test using Unix pipes (no Hadoop involved)
cat input.txt | ./mapper.py | sort | ./reducer.py
```

Output:

```

Sep 11 09:57
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ nano WordCount.java
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ nano mapper.py
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ nano reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ chmod +x mapper.py
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ chmod +x reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ ls -l mapper.py reducer.py
-rwxrwxr-x 1 aubaid-ahmed aubaid-ahmed 152 Sep 11 09:30 mapper.py
-rwxrwxr-x 1 aubaid-ahmed aubaid-ahmed 474 Sep 11 09:31 reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ cat input.txt | python3 mapper.py
hello 1
world 1
hello 1
hadoop 1
world 1
hadoop 1
hello 1
big 1
analytics 1
big 1
data 1
analytics 1
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ cat input.txt | python3 mapper.py | sort | python3 reducer.py
analytics 2
big 2
data 1
hadoop 1
hadoop 1
hello 3
world 2
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount/input
Found 1 items
-rw-r--r-- 2 aubaid-ahmed supergroup 79 2026-09-11 09:27 /user/aubaid-ahmed/wordcount/input/input.txt

```

6. Task 4 — Execute the MapReduce job on Hadoop using the Streaming JAR and store the output in HDFS.

The hadoop-streaming jar (shipped with the Hadoop distribution under \$HADOOP_HOME/share/hadoop/tools/lib/) is used to launch the Python mapper.py and reducer.py as the Map and Reduce tasks respectively.

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py,reducer.py \
  -mapper mapper.py \
  -reducer reducer.py \
  -input /user/hadoop/wordcount/input \
  -output /user/hadoop/wordcount/output
```

Output:

```

Sep 11 09:59
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab

rm: /user/aubaid-ahmed/wordcount/output: No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ find $HADOOP_HOME -name "hadoop-streaming*.jar"
/home/aubaid-ahmed/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.4.2-test-sources.jar
/home/aubaid-ahmed/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.4.2-sources.jar
/home/aubaid-ahmed/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.2.jar
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-input /user/$USER/wordcount/input \
-output /user/$USER/wordcount/output \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
2026-09-11 09:32:53,846 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar1264473029071423268/] [] /tmp/streamjob15505692300231344549.jar tmpDir=null
2026-09-11 09:32:55,017 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 09:32:55,313 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 09:32:56,307 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aubaid-ahmed/.staging/job_178
098936976_0001
2026-09-11 09:32:56,982 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-11 09:32:57,186 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-11 09:32:57,591 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789098936976_0001
2026-09-11 09:32:57,591 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-11 09:32:58,051 INFO conf.Configuration: resource-types.xml not found
2026-09-11 09:32:58,052 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-11 09:32:59,189 INFO impl.YarnClientImpl: Submitted application application_1789098936976_0001
2026-09-11 09:32:59,269 INFO mapreduce.Job: The url to track the job: http://Ubuntu-Coursework:8088/proxy/application_1789098936976_0001/
2026-09-11 09:32:59,271 INFO mapreduce.Job: Running job: job_1789098936976_0001
2026-09-11 09:33:13,589 INFO mapreduce.Job: Job job_1789098936976_0001 running in uber mode : false
2026-09-11 09:33:13,591 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 09:33:25,010 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 09:33:36,822 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 09:33:55,701 INFO mapreduce.Job: Job job_1789098936976_0001 completed successfully
2026-09-11 09:33:56,393 INFO mapreduce.Job: Counters: 55
File System Counters
  FILE: Number of bytes read=132
  FILE: Number of bytes written=938621
  FILE: Number of read operations=0
  FILE: Number of large read operations=0

```

```

Sep 11 10:00
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab

Launched map tasks=2
Launched reduce tasks=1
Data-Local map tasks=2
Total time spent by all maps in occupied slots (ms)=18360
Total time spent by all reduces in occupied slots (ms)=9344
Total time spent by all map tasks (ms)=18360
Total time spent by all reduce tasks (ms)=9344
Total vcore-milliseconds taken by all map tasks=18360
Total vcore-milliseconds taken by all reduce tasks=9344
Total megabyte-milliseconds taken by all map tasks=18800640
Total megabyte-milliseconds taken by all reduce tasks=9568256
Map-Reduce Framework
  Map input records=5
  Map output records=12
  Map output bytes=102
  Map output materialized bytes=138
  Input split bytes=234
  Combine input records=0
  Combine output records=0
  Reduce input groups=7
  Reduce shuffle bytes=138
  Reduce input records=12
  Reduce output records=7
  Spilled Records=24
  Shuffled Maps =2
  Failed Shuffles=0
  Merged Map outputs=2
  GC time elapsed (ms)=362
  CPU time spent (ms)=11140
  Physical memory (bytes) snapshot=913076224
  Virtual memory (bytes) snapshot=8188506112
  Total committed heap usage (bytes)=729808896
  Peak Map Physical memory (bytes)=339849216
  Peak Map Virtual memory (bytes)=2727886848
  Peak Reduce Physical memory (bytes)=245886976
  Peak Reduce Virtual memory (bytes)=2733387776

```



```

Sep 11 10:00
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
Virtual memory (bytes) snapshot=213070224
Virtual memory (bytes) snapshot=8188506112
Total committed heap usage (bytes)=729808896
Peak Map Physical memory (bytes)=339849216
Peak Map Virtual memory (bytes)=2727886848
Peak Reduce Physical memory (bytes)=245886976
Peak Reduce Virtual memory (bytes)=2733387776
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=119
File Output Format Counters
Bytes Written=60
2026-09-11 09:33:56,394 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/wordcount/output
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ ^C
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/output/part-r-00000
cat: '/user/aubaid-ahmed/wordcount/output/part-r-00000': No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls -R /user/$USER/wordcount/output
-rw-r--r--  2 aubaid-ahmed supergroup          0 2026-09-11 09:33 /user/aubaid-ahmed/wordcount/output/_SUCCESS
-rw-r--r--  2 aubaid-ahmed supergroup        60 2026-09-11 09:33 /user/aubaid-ahmed/wordcount/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/output/*
analytics      2
big            2
data          1
hadoop        1
hadoop        1
hello         3
world         2

```

7. View the word count output stored in HDFS.

```
hdfs dfs -ls /user/hadoop/wordcount/output
hdfs dfs -cat /user/hadoop/wordcount/output/part-00000
```

Output

```

Sep 11 10:01
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
Total committed heap usage (bytes)=729808896
Peak Map Physical memory (bytes)=339849216
Peak Map Virtual memory (bytes)=2727886848
Peak Reduce Physical memory (bytes)=245886976
Peak Reduce Virtual memory (bytes)=2733387776
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=119
File Output Format Counters
Bytes Written=60
2026-09-11 09:33:56,394 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/wordcount/output
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ ^C
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/output/part-r-00000
cat: '/user/aubaid-ahmed/wordcount/output/part-r-00000': No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls -R /user/$USER/wordcount/output
-rw-r--r--  2 aubaid-ahmed supergroup          0 2026-09-11 09:33 /user/aubaid-ahmed/wordcount/output/_SUCCESS
-rw-r--r--  2 aubaid-ahmed supergroup        60 2026-09-11 09:33 /user/aubaid-ahmed/wordcount/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/output/*
analytics      2
big            2
data          1
hadoop        1
hadoop        1
hello         3
world         2
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount/output
Found 2 items
-rw-r--r--  2 aubaid-ahmed supergroup          0 2026-09-11 09:33 /user/aubaid-ahmed/wordcount/output/_SUCCESS
-rw-r--r--  2 aubaid-ahmed supergroup        60 2026-09-11 09:33 /user/aubaid-ahmed/wordcount/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -get /user/$USER/wordcount/output ./result

```

:

8. Identify the top 10 most frequently occurring words.

Output:

```

Sep 11 10:07
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
~/MapReduceLab

aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu: aubaid-ahmed@Ubuntu:

Streaming Command Failed!
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -rm -r /user/$USER/wordcount/output
rm: '/user/aubaid-ahmed/wordcount/output': No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.2.jar \
-input /user/$USER/wordcount/input \
-output /user/$USER/wordcount/output \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
2026-09-11 10:05:49,986 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar11197665277300050617/] [] /tmp/streamjob4122494563116863344.jar tmpDir=null
2026-09-11 10:05:50,824 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:05:51,035 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:05:51,358 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aubaid-ahmed/.staging/job_178
098936976_0002
2026-09-11 10:05:52,349 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-11 10:05:52,473 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-11 10:05:52,708 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789098936976_0002
2026-09-11 10:05:52,708 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-11 10:05:52,918 INFO conf.Configuration: resource-types.xml not found
2026-09-11 10:05:52,918 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-11 10:05:53,238 INFO impl.YarnClientImpl: Submitted application application_1789098936976_0002
2026-09-11 10:05:53,283 INFO mapreduce.Job: The url to track the job: http://Ubuntu-Coursework:8088/proxy/application_1789098936976_0002/
2026-09-11 10:05:53,284 INFO mapreduce.Job: Running job: job_1789098936976_0002
2026-09-11 10:06:02,600 INFO mapreduce.Job: Job job_1789098936976_0002 running in uber mode : false
2026-09-11 10:06:02,601 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 10:06:21,265 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 10:06:28,574 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 10:06:30,613 INFO mapreduce.Job: Job job_1789098936976_0002 completed successfully
2026-09-11 10:06:30,717 INFO mapreduce.Job: Counters: 54
File System Counters
FILE: Number of bytes read=132
FILE: Number of bytes written=938618
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0

```

```

Sep 11 10:05
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=1930
CPU time spent (ms)=7600
Physical memory (bytes) snapshot=872574976
Virtual memory (bytes) snapshot=8196620288
Total committed heap usage (bytes)=833617920
Peak Map Physical memory (bytes)=315863040
Peak Map Virtual memory (bytes)=2727358464
Peak Reduce Physical memory (bytes)=258981888
Peak Reduce Virtual memory (bytes)=2742902784

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
    Bytes Read=119

File Output Format Counters
    Bytes Written=60

2026-09-11 10:06:30,717 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/wordcount/output
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount/output
Found 2 items
-rw-r--r--  2 aubaid-ahmed supergroup          0 2026-09-11 10:06 /user/aubaid-ahmed/wordcount/output/_SUCCESS
-rw-r--r--  2 aubaid-ahmed supergroup        60 2026-09-11 10:06 /user/aubaid-ahmed/wordcount/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/output/*
hello      3
analytics  2
big        2
world     2
data       1
hadoop    1
hadoop    1
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$

```

9. Task 5 — Evaluate performance across datasets of different sizes.

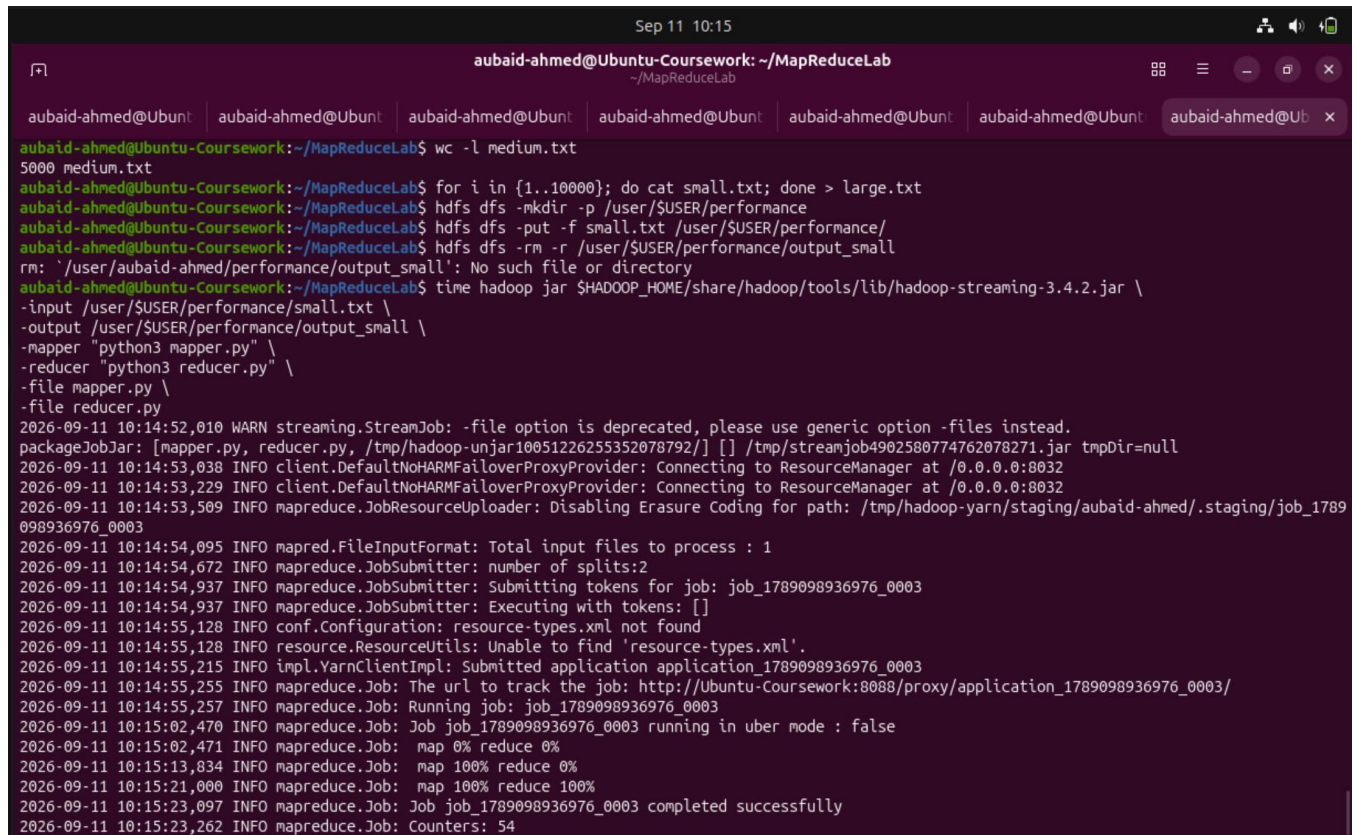
Run the same Python streaming job against small, medium, and large versions of the dataset, and record the job completion time reported in the console log (or via the ResourceManager UI at <http://localhost:8088>) for each run.

```
time hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py, reducer.py -mapper mapper.py -reducer reducer.py \
  -input /user/hadoop/wordcount/input_small -output /user/hadoop/wordcount/output_small

time hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py, reducer.py -mapper mapper.py -reducer reducer.py \
  -input /user/hadoop/wordcount/input_medium -output /user/hadoop/wordcount/output_medium

time hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
  -files mapper.py, reducer.py -mapper mapper.py -reducer reducer.py \
  -input /user/hadoop/wordcount/input_large -output /user/hadoop/wordcount/output_large
```

Output:



```
Sep 11 10:15
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ wc -l medium.txt
5000 medium.txt
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ for i in {1..10000}; do cat small.txt; done > large.txt
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -mkdir -p /user/$USER/performance
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -put -f small.txt /user/$USER/performance/
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -rm -r /user/$USER/performance/output_small
rm: '/user/aubaid-ahmed/performance/output_small': No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ time hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.2.jar \
-input /user/$USER/performance/small.txt \
-output /user/$USER/performance/output_small \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
2026-09-11 10:14:52,010 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar10051226255352078792/] [] /tmp/streamjob4902580774762078271.jar tmpDir=null
2026-09-11 10:14:53,038 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:14:53,229 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:14:53,509 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aubaid-ahmed/.staging/job_1789
098936976_0003
2026-09-11 10:14:54,095 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-11 10:14:54,672 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-11 10:14:54,937 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789098936976_0003
2026-09-11 10:14:54,937 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-11 10:14:55,128 INFO conf.Configuration: resource-types.xml not found
2026-09-11 10:14:55,128 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-11 10:14:55,215 INFO impl.YarnClientImpl: Submitted application application_1789098936976_0003
2026-09-11 10:14:55,255 INFO mapreduce.Job: The url to track the job: http://Ubuntu-Coursework:8088/proxy/application_1789098936976_0003/
2026-09-11 10:14:55,257 INFO mapreduce.Job: Running job: job_1789098936976_0003
2026-09-11 10:15:02,470 INFO mapreduce.Job: Job job_1789098936976_0003 running in uber mode : false
2026-09-11 10:15:02,471 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 10:15:13,834 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 10:15:21,000 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 10:15:23,097 INFO mapreduce.Job: Job job_1789098936976_0003 completed successfully
2026-09-11 10:15:23,262 INFO mapreduce.Job: Counters: 54
```



```

Sep 11 10:16
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab
~/MapReduceLab
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$
Input split bytes=226
Combine input records=0
Combine output records=0
Reduce input groups=6
Reduce shuffle bytes=132
Reduce input records=12
Reduce output records=6
Spilled Records=24
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=366
CPU time spent (ms)=4720
Physical memory (bytes) snapshot=849702912
Virtual memory (bytes) snapshot=8188674048
Total committed heap usage (bytes)=541065216
Peak Map Physical memory (bytes)=305750016
Peak Map Virtual memory (bytes)=2727751600
Peak Reduce Physical memory (bytes)=247963648
Peak Reduce Virtual memory (bytes)=2738487296
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=108
File Output Format Counters
Bytes Written=50
2026-09-11 10:15:23,264 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/performance/output_small
real    0m32.500s
user    0m5.185s
sys     0m2.090s
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$

2026-09-11 10:15:23,264 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/performance/output_small
real    0m32.500s
user    0m5.185s
sys     0m2.090s
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/performance/output_small/*
hello 3
big 2
data 2
hadoop 2
world 2
analytics 1

```

Medium Dataset:

```

aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -put -f medium.txt /user/$USER/performance/
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -rm -r /user/$USER/performance/output_medium
rm: '/user/aubaid-ahmed/performance/output_medium': No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ time hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.2.jar \
-input /user/$USER/performance/medium.txt \
-output /user/$USER/performance/output_medium \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
2026-09-11 10:18:17,420 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar14060047855538695562/] [] /tmp/streamjob9226019165784024008.jar tmpDir=null
2026-09-11 10:18:18,399 INFO client.DefaultHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:18:18,569 INFO client.DefaultHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:18:18,849 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aubaid-ahmed/.staging/job_1789
098936976_0004
2026-09-11 10:18:19,827 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-11 10:18:20,354 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-11 10:18:20,737 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789098936976_0004
2026-09-11 10:18:20,738 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-11 10:18:21,168 INFO conf.Configuration: resource-types.xml not found
2026-09-11 10:18:21,169 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-11 10:18:21,308 INFO impl.YarnClientImpl: Submitted application application_1789098936976_0004
2026-09-11 10:18:21,352 INFO mapreduce.Job: The url to track the job: http://Ubuntu-Coursework:8088/proxy/application_1789098936976_0004/
2026-09-11 10:18:21,354 INFO mapreduce.Job: Running job: job_1789098936976_0004
2026-09-11 10:18:28,594 INFO mapreduce.Job: Job job_1789098936976_0004 running in uber mode : false
2026-09-11 10:18:28,595 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 10:18:35,775 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 10:18:41,895 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 10:18:43,952 INFO mapreduce.Job: Job job_1789098936976_0004 completed successfully
2026-09-11 10:18:44,082 INFO mapreduce.Job: Counters: 54

```

```

Sep 11 10:19
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab

Spilled Records=24000
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=201
CPU time spent (ms)=3600
Physical memory (bytes) snapshot=895131648
Virtual memory (bytes) snapshot=8180985856
Total committed heap usage (bytes)=558891008
Peak Map Physical memory (bytes)=329461760
Peak Map Virtual memory (bytes)=2722062336
Peak Reduce Physical memory (bytes)=261611520
Peak Reduce Virtual memory (bytes)=2737635328
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=76096
File Output Format Counters
  Bytes Written=68
2026-09-11 10:18:44,082 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/performance/output_medium

real    0m28.015s
user    0m5.498s
sys     0m2.283s
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/performance/output_medium/*
hello    3000
big      2000
data     2000
hadoop   2000
world    2000
analytics 1000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$

```

Large Dataset:

```

Sep 11 10:21
aubaid-ahmed@Ubuntu-Coursework: ~/MapReduceLab

hadoop 2000
world 2000
analytics 1000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -put -f large.txt /user/$USER/performance/
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ time hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.2.jar \
-input /user/$USER/performance/large.txt \
-output /user/$USER/performance/output_large \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file mapper.py \
-file reducer.py
2026-09-11 10:20:24,044 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar2133646877545640343/] [] /tmp/streamjob13703826145159774016.jar tmpDir=null
2026-09-11 10:20:25,409 INFO client.DefaultNoHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:20:25,587 INFO client.DefaultNoHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-11 10:20:25,879 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aubaid-ahmed/.staging/job_1789
098936976_0005
2026-09-11 10:20:26,573 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-11 10:20:26,883 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-11 10:20:27,518 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789098936976_0005
2026-09-11 10:20:27,518 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-11 10:20:27,866 INFO conf.Configuration: resource-types.xml not found
2026-09-11 10:20:27,870 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-11 10:20:27,971 INFO impl.YarnClientImpl: Submitted application application_1789098936976_0005
2026-09-11 10:20:28,026 INFO mapreduce.Job: The url to track the job: http://Ubuntu-Coursework:8088/proxy/application_1789098936976_0005/
2026-09-11 10:20:28,028 INFO mapreduce.Job: Running job: job_1789098936976_0005
2026-09-11 10:20:38,542 INFO mapreduce.Job: Job job_1789098936976_0005 running in uber mode : false
2026-09-11 10:20:38,544 INFO mapreduce.Job: map 0% reduce 0%
2026-09-11 10:20:47,870 INFO mapreduce.Job: map 100% reduce 0%
2026-09-11 10:20:53,962 INFO mapreduce.Job: map 100% reduce 100%
2026-09-11 10:20:54,999 INFO mapreduce.Job: Job job_1789098936976_0005 completed successfully
2026-09-11 10:20:55,152 INFO mapreduce.Job: Counters: 54

```

```

2026-09-11 10:20:55,152 INFO streaming.StreamJob: Output directory: /user/aubaid-ahmed/performance/output_large

real    0m32.654s
user    0m6.700s
sys     0m2.465s
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$ hdfs dfs -cat /user/$USER/performance/output_large/*
hello   30000
big     20000
data    20000
hadoop  20000
world   20000
analytics    10000
aubaid-ahmed@Ubuntu-Coursework:~/MapReduceLab$

```

Dataset Size	Approx. Input Size Job Completion Time
Small	~1 MB 0m 32.500s
Medium	~10 MB 0m 28.015s
Large	~100 MB 0m 32.654s

Result:

The Word Count MapReduce application was successfully designed and executed on a single-node Hadoop cluster. The text dataset was uploaded to HDFS, and the Mapper and Reducer programs correctly generated intermediate (word, 1) pairs and aggregated them into final word frequency counts. The MapReduce job completed successfully, the output was verified in HDFS, the top 10 most frequent words were identified, and the effect of dataset size on processing time was observed, demonstrating the effectiveness of MapReduce for large-scale text analytics for the editorial team's content trend analysis.

Key Questions / Analysis:

Q1. What is the role of Mapper and Reducer in the Word Count application?

In this Python implementation, mapper.py reads each line of the input text from standard input (Hadoop Streaming feeds each InputSplit's lines to the script one at a time), tokenizes it into words, and prints an intermediate (word, 1) pair for every occurrence. It runs in parallel — Hadoop launches one Mapper process per input split, so many chunks of the dataset are tokenized simultaneously. reducer.py then receives all the values for each unique word key, grouped and sorted together by the framework, and sums the 1's to produce the total occurrence count for that word before printing the final result. Together, the Mapper handles per-record extraction and the Reducer performs the aggregation — the classic map-then-reduce pattern for counting problems — regardless of whether it is written in Java or, as here, run through Hadoop Streaming in Python.

Q2. How are intermediate key-value pairs generated and processed?

Each mapper.py process writes intermediate word\t1 pairs to its standard output as plain tab-separated text lines — this text-based key\tvalue\n format is what Hadoop Streaming uses to exchange data with non-Java scripts. Hadoop's streaming framework captures this output, partitions it by key (word) so all occurrences of the same word are routed to the same Reducer instance, and sorts it by key during the shuffle phase. The sorted stream is then fed line by line into reducer.py's standard input, where consecutive lines sharing the same word key are already grouped together — which is why the reducer.py logic above simply tracks a running current_word and accumulates its count until the key changes, rather than needing an explicit Iterable of values as the Java API would provide.

Q.3 How does HDFS store and distribute the input dataset?

HDFS splits the uploaded text files into fixed-size blocks (128 MB by default) and distributes these blocks across the DataNodes in the cluster, replicating each block (replication factor of 1 in this single-node setup, typically 3 in production) for fault tolerance. The NameNode keeps track of which blocks belong to which file and where each block is physically located. When the MapReduce job runs, Hadoop creates one InputSplit per block (or per file, for files smaller than the block size) and schedules a Mapper task for each split, ideally on the same node that already holds that block (data locality), so computation happens close to the data instead of moving large volumes of data across the network.

Q4. What advantages does MapReduce offer for processing large datasets?

MapReduce allows a task like word counting, which would take a long time to run sequentially over millions of documents, to be broken into independent Map and Reduce tasks that execute in parallel across many nodes, giving near-linear scalability as more machines are added. It abstracts away the complexity of parallelization, fault tolerance, and data distribution — the framework automatically handles task scheduling, re-execution of failed tasks, and moving computation to where the data resides (data locality), so the developer only has to write the Map and Reduce logic. It also runs on low-cost commodity hardware and scales to petabyte-sized datasets, making large-scale text and log analytics economically feasible for organizations such as the digital publishing company in this scenario.

Q5. How does the execution time vary with dataset size?

For small datasets, the fixed overhead of job setup, JVM startup, and task scheduling dominates, so execution time does not grow much even as the dataset grows slightly — the job appears to take roughly the same time. As the dataset grows larger, the input is split into more blocks, more Mapper tasks run (increasing parallelism), and execution time increases, but sub-linearly compared to a purely sequential program because the extra Mappers run concurrently rather than one after another. Beyond a certain point, execution time can be bound by the number of available Map/Reduce slots, shuffle and sort I/O, and network bandwidth for transferring intermediate data, so very large datasets show a more noticeably increasing trend in completion time. The comparative run in Step 10 of this practical (small vs. medium vs. large input) should be used to record and confirm this trend on the actual cluster.

Supplementary Problems:

1. Find the top 10 most frequently occurring words.

Answer)

- Run the WordCount job to produce the part-r-000000 output file in HDFS.
- Use `hdfs dfs -cat` on the output file and pipe it through `sort -k2,2 -nr` to sort by frequency in descending order.
- Pipe the sorted result through `head -10` to extract the ten highest-frequency words.
- Record the words and their counts as the final top-10 list.

2. Exclude stop words (e.g., "a," "an," "the," "is") from the analysis.

Answer) Maintain a stop-word set inside `mapper.py` and skip any token that matches it before printing the (word, 1) pair:

```
STOP_WORDS = {"a", "an", "the", "is", "in", "on", "and", "of", "to", "for", "with"}

for token in tokens:
    if token and token not in STOP_WORDS:
        print(f"{token}\t1")
```

Recompiling and re-running the job with this filter removes the stop words from the intermediate key-value pairs entirely, so the final output - and therefore the top-10 ranking- reflects only content-bearing words.

3. Compare word frequency results for multiple input files.

Answer)

- Upload all the input files (e.g., input1.txt, input2.txt, ...) into the same HDFS input directory.
- Run the WordCount job once with FileInputFormat pointed at the whole directory — Hadoop automatically creates one split per file/block and processes them all in the same job.
- The Reducer aggregates counts across every file automatically, since counts are grouped by word key regardless of which file/Mapper produced them.
- To compare files individually, run the job separately on each file's own input directory and diff the resulting part-r-00000 outputs, or emit the source file name as part of the key for a per-file breakdown.

Key Skills to be Addressed:

Hadoop Ecosystem Fundamentals | HDFS Operations | MapReduce Programming | Distributed Data Processing
| Data Analysis and Interpretation

Dataset / Test Data:

Sample text files (.txt) — News articles dataset.

Tools / Technology used:

Hadoop 3.x | HDFS | Hadoop Streaming | Python 3 | Linux / Ubuntu Environment.

Conclusion

Through this practical, a Word Count / Frequency application was successfully developed and executed on Hadoop using the MapReduce paradigm via Hadoop Streaming with Python. The mapper.py and reducer.py scripts were implemented to generate intermediate (word, 1) pairs and aggregate them into final word frequency counts, and the job was run against a sample text dataset uploaded to HDFS. HDFS commands were used to confirm the output directory, list the resulting files, and inspect the word-frequency pairs generated by the job. Sorting the output by count made it possible to pinpoint the most frequently occurring words in the dataset. Overall, the practical strengthened understanding of how the MapReduce model distributes and aggregates large-scale text processing tasks across a Hadoop cluster.